

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт компьютерных наук

Кафедра автоматизированных систем управления

Практическая работа №10

по дисциплине

«Управление ИТ-проектами»

Студент М-РИТ-25-1 _____ Станиславчук С.М.

Руководитель

К.т.н., доцент _____ Ведищев В.В.

Липецк, 2026

Содержание

1. Примеры систем командной разработки 3

2. Организация взаимодействия 3

3. Контроль версий 4

1. Примеры систем командной разработки

Для организации совместной разработки программных продуктов используются специализированные системы, обеспечивающие хранение кода, управление изменениями и координацию участников проекта. К таким системам относятся GitLab, GitHub, Bitbucket, а также системы управления задачами, такие как Jira.

В рамках данного проекта основным инструментом является система контроля версий Git, размещённая на платформе GitHub. Она обеспечивает централизованное хранение исходного кода, управление ветками и поддержку совместной работы разработчиков.

2. Организация взаимодействия

Взаимодействие внутри команды организуется через систему контроля версий и систему управления задачами. Каждая задача фиксируется в трекере, после чего назначается ответственному исполнителю.

Разработка ведётся в отдельных ветках, соответствующих функциональности или исправлению ошибок. Бэкенд- и фронтенд-разработчики работают параллельно, согласуя интерфейсы взаимодействия. После завершения работы изменения проходят процедуру проверки, что позволяет выявить ошибки до интеграции в основную ветку.

Специалист по информационной безопасности участвует в проверке изменений, связанных с обработкой данных и механизмами защиты. Тестировщик проверяет реализованный функционал и формирует отчёты об ошибках. Системный администратор обеспечивает работоспособность инфраструктуры и участвует в развёртывании системы.

Таким образом, взаимодействие строится на принципах распределённой разработки с централизованным контролем изменений.

3. Контроль версий

Контроль версий осуществляется с использованием Git. Основная ветка проекта содержит стабильную версию системы, в то время как разработка ведётся в отдельных ветках, создаваемых для реализации конкретных задач.

Используется модель ветвления, включающая ветки `main`, `develop`, `feature/*`, `bugfix/*` и `hotfix/*`. Это позволяет изолировать изменения, упростить их тестирование и контролировать процесс интеграции.

Каждое изменение фиксируется в виде коммита с описанием выполненной работы. Перед объединением изменений в основную ветку проводится проверка кода, а также тестирование. Для обозначения стабильных версий используются теги, соответствующие принятой схеме версионирования.

Данный подход обеспечивает отслеживаемость изменений, возможность возврата к предыдущим версиям и повышение качества разработки программного продукта.